# Advanced NumPy - Assignment

Solve every question using NumPy only - no Python `for` loops unless a question explicitly allows one. Where a question asks you to predict something first, write the prediction as a comment before running the code. Print the shape of your result whenever an answer involves reshaping, stacking or matrix multiplication. Searching the documentation is allowed and encouraged.

**Q1.** A class has 4 students and 3 subjects. Their marks are given below. The examiner announces a subject-wise bonus of 5, 2 and 8 marks respectively. Add the bonus to every student in one line using broadcasting, then cap any mark above 100 back to 100 and report how many marks had to be capped.

```
marks = np.array([[88, 96, 95],
                  [72, 99, 91],
                  [98, 85, 97],
                  [60, 75, 80]])
```

**Q2.** Create a 5x5 matrix with a border of 1s and 0s inside, without writing the matrix out by hand.

**Q3.** Without running the code, predict the shape of `np.arange(30).reshape(5, -1).T`. Write your prediction as a comment, then verify it. Also explain in a comment why `np.arange(30).reshape(4, -1)` fails.

**Q4.** Two arrays of size 4 are given. First join them with `hstack` and print the shape. Then, using the same two arrays, build a proper (4, 2) two-column table. In a comment, state why the first attempt did not produce two columns.

```
heights = np.array([172, 158, 181, 166])
weights = np.array([68, 52, 84, 59])
```

**Q5.** Given a 1D array of 12 monthly sales figures, reshape it into 4 quarters of 3 months each and find which quarter (index) had the highest average sales, along with that average.

```
sales = np.array([120, 135, 98, 160, 175, 190, 88, 92, 105, 210, 198, 230])
```

**Q6.** Build a 3x3 multiplication table from the array below using broadcasting only. Do not use loops and do not use `np.outer` — reshape one of the operands so that broadcasting produces the grid.

```
a = np.array([2, 3, 4])
```

**Q7.** Create an array of numbers from 1 to 20 and replace every multiple of 3 with -1.

**Q8.** For the matrix below, scale every column into the 0 to 1 range using column-wise min-max normalization `(x - col_min) / (col_max - col_min)`. Use broadcasting only — no loops. Verify that each column of the result has a minimum of 0 and a maximum of 1.

```
X = np.array([[ 50, 1200,  3],
              [ 80, 1500,  9],
              [ 65,  900,  5],
              [ 95, 2000,  7]])
```

**Q9.** Award medals to the scores below using nested `np.where`: 'Gold' for 90 and above, 'Silver' for 75 and above, 'Bronze' for 60 and above, and 'None' otherwise. Then count how many students received each medal.

```
scores = np.array([94, 61, 77, 45, 90, 88, 59, 72])
```

**Q10.** Given the matrix below, use `np.where` with a single condition to obtain the row and column indices of every element greater than 50, print those elements, and then replace them with 0 in a copy of the matrix while leaving the original untouched.

```
m = np.array([[10, 80, 45],
              [55, 20, 99],
              [70, 33, 12]])
```

**Q11.** Three sensors each recorded 5 readings. Stack them so that each sensor becomes a row, and separately stack them so that each sensor becomes a column. Print both shapes and confirm that one is the transpose of the other.

```
s1 = np.array([12, 15, 11, 19, 14])
s2 = np.array([22, 18, 25, 21, 20])
s3 = np.array([31, 35, 29, 33, 30])
```

**Q12.** Create a random 6x6 matrix with `seed = 1` and values between 1 and 100, then find the row-wise maximum and the column-wise minimum.

**Q13.** A and B are both 2x3 matrices. Compute `A @ B.T` and `A.T @ B`. Print the shape of each result and explain in a comment why both multiplications are legal while `A @ B` is not.

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])
B = np.array([[7, 8, 9],
              [1, 0, 2]])
```

**Q14.** Build a complete multi-step pipeline. Starting from the two arrays below: convert them into column vectors, join them into a single two-column table, attach a third column holding each row's total, then multiply the three-column table by `w = [0.5, 0.2, 0.3]` to obtain one score per row. Finally use `np.where` to mark each score as 1 if it is above the average score and 0 otherwise. Report the shape of every intermediate array.

```
hours = np.array([2, 8, 5, 10, 3, 7])
attendance = np.array([55, 92, 70, 98, 60, 85])
```

**Q15.** Apply min-max normalization to scale any array into the 0 to 1 range using `(x - min) / (max - min)`.

**Q16.** Take the array below and create two 1D versions of it — one with `flatten()` and one with `ravel()`. Change the first element of each, then print the original matrix. Explain the difference you observe in a comment.

```
m = np.array([[4, 8, 15],
              [16, 23, 42]])
```

**Q17.** 28 daily temperature readings are given. Reshape them into 4 weeks of 7 days, then: find each week's average, identify the hottest week, and count how many individual days crossed 35 degrees.

```
np.random.seed(7)
temps = np.random.randint(15, 41, 28)
```

**Q18.** Create two random 3x3 matrices A and B, compute `A @ B` and `B @ A`, and check whether the two results are equal. State your conclusion about matrix multiplication in a comment.

**Q19.** Given the marks matrix below (5 students x 4 subjects), first replace every mark below 35 with 35, then compute each student's total, and finally print the index of the topper along with their total.

```
marks = np.array([[45, 28, 67, 80],
                  [33, 90, 55, 41],
                  [72, 66, 30, 58],
                  [88, 79, 91, 84],
                  [25, 40, 38, 52]])
```

**Q20.** Start from `arr = np.arange(1, 11)`. Create one variable using a plain slice `arr[2:6]` and another using `arr[2:6].copy()`. Change the first element of each to 999, then print the original array and explain in a comment which change survived and why.

**Q21.** Explore `np.diag` in the NumPy documentation and use it to extract the diagonal of a 4x4 matrix. Then use the same function in its second role — to build a diagonal matrix from a 1D array.